Kubernetes Networking Demonstration Guide

1. Kubernetes Network Model Demonstration

Concept Overview

```
# The Kubernetes network model has 4 key requirements:
# 1. Pods can communicate with all other pods without NAT
# 2. Nodes can communicate with all pods without NAT
# 3. The IP a pod sees itself as is the same IP others see it as
# 4. Container-to-container communication within a pod via localhost
```

Demo: Pod-to-Pod Communication

```yaml
# Create two demo pods
# pod1.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  labels:
    app: demo
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80

# pod2.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod2
spec:
  containers:
  - name: busybox
    image: busybox
    command: ['sh', '-c', 'sleep 3600']
```

```bash
# Apply the pods
kubectl apply -f pod1.yaml
kubectl apply -f pod2.yaml
```

```
# Get pod IPs
kubectl get pods -o wide

# Test connectivity from pod2 to pod1
kubectl exec -it pod2 -- wget -qO- <POD1_IP>
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl get pods -o wide
NAME                              READY   STATUS    RESTARTS   AGE     IP               NODE              NOMINATED NODE   READINESS GATES
example-gateway-858d8c444b-nwbx4   1/1     Running   0          2m34s   192.168.22.150   ip-172-31-33-209   <none>           <none>
pod1                              1/1     Running   0          14s     192.168.108.147  ip-172-31-35-239   <none>           <none>
pod2                              1/1     Running   0          6s      192.168.108.131  ip-172-31-35-239   <none>           <none>
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -it pod2 -- wget -qO- 192.168.108.147
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@ip-172-31-42-185:/home/ubuntu/demo#
```

2. CNI Plugins Demonstration

Check Current CNI Plugin

```
# Check which CNI is installed
kubectl get pods -n kube-system | grep -E 'calico|flannel|weave|cilium'
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl get pods -n kube-system | grep -E 'calico|flannel|weave|cilium'
calico-kube-controllers-b45f49df6-69zc9     1/1     Running   37 (25m ago)   33d
calico-node-jgqkj                           1/1     Running   37 (25m ago)   33d
calico-node-mkr9k                           1/1     Running   37 (25m ago)   33d
calico-node-njmmv                           1/1     Running   37 (25m ago)   33d
root@ip-172-31-42-185:/home/ubuntu/demo#
```

```
# Check CNI configuration
ls /etc/cni/net.d/
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# ls /etc/cni/net.d/
10-calico.conflist  calico-kubeconfig
```

```
# View CNI config (example)
cat /etc/cni/net.d/10-calico.conflist
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# cat /etc/cni/net.d/10-calico.conflist
{
  "name": "k8s-pod-network",
  "cniVersion": "0.3.1",
  "plugins": [
    {
      "type": "calico",
      "log_level": "info",
      "log_file_path": "/var/log/calico/cni/cni.log",
      "datastore_type": "kubernetes",
      "nodename": "ip-172-31-42-185",
      "mtu": 0,
      "ipam": {
          "type": "calico-ipam"
      },
      "policy": {
          "type": "k8s"
      },
      "kubernetes": {
          "kubeconfig": "/etc/cni/net.d/calico-kubeconfig"
      }
    },
    {
      "type": "portmap",
      "snat": true,
      "capabilities": {"portMappings": true}
    },
    {
      "type": "bandwidth",
      "capabilities": {"bandwidth": true}
    }
  ]
}root@ip-172-31-42-185:/home/ubuntu/demo#
```

Install Calico CNI (Example)

```
# Download Calico manifest
curl https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/calico.yaml
-O

# Apply Calico
kubectl apply -f calico.yaml

# Verify installation
kubectl get pods -n kube-system -l k8s-app=calico-node
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl get pods -n kube-system -l k8s-app=calico-node
NAME                READY   STATUS    RESTARTS       AGE
calico-node-jgqkj   1/1     Running   37 (28m ago)   33d
calico-node-mkr9k   1/1     Running   37 (27m ago)   33d
calico-node-njmmv   1/1     Running   37 (27m ago)   33d
root@ip-172-31-42-185:/home/ubuntu/demo#
```

3. DNS in Kubernetes Demonstration

CoreDNS Exploration

```
# Check CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl get pods -n kube-system -l k8s-app=kube-dns
NAME                       READY   STATUS    RESTARTS       AGE
coredns-66bc5c9577-795x2   1/1     Running   37 (28m ago)   33d
coredns-66bc5c9577-qm2jt   1/1     Running   37 (28m ago)   33d
root@ip-172-31-42-185:/home/ubuntu/demo#
```

```
# View CoreDNS configuration
kubectl get configmap coredns -n kube-system -o yaml
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl get configmap coredns -n kube-system -o yaml
apiVersion: v1
data:
  Corefile: |
    .:53 {
        errors
        health {
           lameduck 5s
        }
        ready
        kubernetes cluster.local in-addr.arpa ip6.arpa {
           pods insecure
           fallthrough in-addr.arpa ip6.arpa
           ttl 30
        }
        prometheus :9153
        forward . /etc/resolv.conf {
           max_concurrent 1000
        }
        cache 30 {
           disable success cluster.local
           disable denial cluster.local
        }
        loop
        reload
        loadbalance
    }
kind: ConfigMap
metadata:
  creationTimestamp: "2026-03-10T03:41:10Z"
  name: coredns
  namespace: kube-system
  resourceVersion: "261"
  uid: ac3cccaa-6808-4b50-8deb-f439649b897a
```

```
# Check DNS service
kubectl get svc -n kube-system kube-dns
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl get svc -n kube-system kube-dns
NAME       TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)                  AGE
kube-dns   ClusterIP   10.96.0.10   <none>        53/UDP,53/TCP,9153/TCP   33d
```

DNS Resolution Demo

```yaml
# Create a service
# demo-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: default
spec:
  selector:
    app: demo
  ports:
  - port: 80
    targetPort: 80
```

```bash
# Apply service
kubectl apply -f demo-service.yaml

# Create a test pod
kubectl run dnstest --image=busybox --restart=Never -- sleep 3600

# Test DNS resolution
kubectl exec -it dnstest -- nslookup my-service
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl run dnstest --image=busybox --restart=Never -- sleep 3600
pod/dnstest created
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -it dnstest -- nslookup my-service
Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   my-service.default.svc.cluster.local
Address: 10.99.251.88

** server can't find my-service.svc.cluster.local: NXDOMAIN

** server can't find my-service.cluster.local: NXDOMAIN

** server can't find my-service.svc.cluster.local: NXDOMAIN


** server can't find my-service.cluster.local: NXDOMAIN

** server can't find my-service.ap-south-1.compute.internal: NXDOMAIN

** server can't find my-service.ap-south-1.compute.internal: NXDOMAIN

command terminated with exit code 1
```

```bash
kubectl exec -it dnstest -- nslookup my-service.default
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -it dnstest -- nslookup my-service.default
Server:         10.96.0.10
Address:        10.96.0.10:53

** server can't find my-service.default: NXDOMAIN

** server can't find my-service.default: NXDOMAIN

command terminated with exit code 1
```

```
kubectl exec -it dnstest -- nslookup my-service.default.svc.cluster.local
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -it dnstest -- nslookup my-service.default.svc.cluster.local
Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   my-service.default.svc.cluster.local
Address: 10.99.251.88
```

```
# Test external DNS
kubectl exec -it dnstest -- nslookup google.com
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -it dnstest -- nslookup google.com
Server:         10.96.0.10
Address:        10.96.0.10:53

Non-authoritative answer:
Name:   google.com
Address: 2404:6800:4009:812::200e

Non-authoritative answer:
Name:   google.com
Address: 142.251.220.14
```

```
# Check DNS configuration in pod
kubectl exec -it dnstest -- cat /etc/resolv.conf
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -it dnstest -- cat /etc/resolv.conf
search default.svc.cluster.local svc.cluster.local cluster.local ap-south-1.compute.internal
nameserver 10.96.0.10
options ndots:5
```

DNS Debugging

```
# Deploy a debug pod with DNS tools
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: dnsutils
spec:
  containers:
  - name: dnsutils
    image: registry.k8s.io/e2e-test-images/jessie-dnsutils:1.3
    command:
      - sleep
      - "3600"
EOF
```

```
# Use dig for detailed DNS queries
kubectl exec -it dnsutils -- dig my-service.default.svc.cluster.local
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -it dnsutils -- dig my-service.default.svc.cluster.local

; <<>> DiG 9.9.5-9+deb8u19-Debian <<>> my-service.default.svc.cluster.local
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6151
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;my-service.default.svc.cluster.local. IN A

;; ANSWER SECTION:
my-service.default.svc.cluster.local. 30 IN A   10.99.251.88

;; Query time: 1 msec
;; SERVER: 10.96.0.10#53(10.96.0.10)
;; WHEN: Sun Apr 12 16:36:54 UTC 2026
;; MSG SIZE  rcvd: 117
```

```
kubectl exec -it dnsutils -- nslookup kubernetes.default
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -it dnsutils -- nslookup kubernetes.default
Server:         10.96.0.10
Address:        10.96.0.10#53

Name:   kubernetes.default.svc.cluster.local
Address: 10.96.0.1
```

4. Network Policies Demonstration

Complete Demo Setup

```yaml
# Create namespaces
apiVersion: v1
kind: Namespace
metadata:
  name: production
---
apiVersion: v1
kind: Namespace
metadata:
  name: development
---
# Backend pod
apiVersion: v1
kind: Pod
metadata:
  name: backend
  namespace: production
  labels:
    app: backend
    tier: api
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
---
# Frontend pod
apiVersion: v1
kind: Pod
metadata:
  name: frontend
  namespace: production
  labels:
    app: frontend
    tier: web
spec:
  containers:
  - name: busybox
    image: busybox
    command: ['sh', '-c', 'sleep 3600']
---
# Test pod (unauthorized)
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
  namespace: production
  labels:
    app: test
spec:
  containers:
```

```yaml
  - name: busybox
    image: busybox
    command: ['sh', '-c', 'sleep 3600']
---
# Network Policy - Allow only frontend to backend
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 80
```

Testing Network Policies

```yaml
# Apply the complete demo
kubectl apply -f network-policy-demo.yaml
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl apply -f network-policy-demo.yaml
namespace/production created
namespace/development created
pod/backend created
pod/frontend created
pod/test-pod created
networkpolicy.networking.k8s.io/backend-policy created
```

```yaml
# Get all pods and their IPs
kubectl get pods -n production -o wide
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl get pods -n production -o wide
NAME       READY   STATUS    RESTARTS   AGE   IP                NODE               NOMINATED NODE   READINESS GATES
backend    1/1     Running   0          27s   192.168.108.188   ip-172-31-35-239   <none>           <none>
frontend   1/1     Running   0          27s   192.168.108.165   ip-172-31-35-239   <none>           <none>
test-pod   1/1     Running   0          27s   192.168.22.190    ip-172-31-33-209   <none>           <none>
```

```
# Test 1: Frontend to Backend (Should work)
kubectl exec -n production frontend -- wget -qO- --timeout=2 <BACKEND_IP>
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -n production frontend -- wget -qO- --timeout=2 192.168.108.188
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@ip-172-31-42-185:/home/ubuntu/demo#
```

```
# Test 2: Test-pod to Backend (Should fail)
kubectl exec -n production test-pod -- wget -qO- --timeout=2 <BACKEND_IP>
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl exec -n production test-pod -- wget -qO- --timeout=2 192.168.108.188
wget: download timed out
command terminated with exit code 1
```

```
# Check network policies
kubectl get networkpolicies -n production
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl get networkpolicies -n production
NAME              POD-SELECTOR   AGE
backend-policy    app=backend    4m8s
```

```
kubectl describe networkpolicy backend-policy -n production
```

```
root@ip-172-31-42-185:/home/ubuntu/demo# kubectl describe networkpolicy backend-policy -n production
Name:         backend-policy
Namespace:    production
Created on:   2026-04-12 16:47:46 +0000 UTC
Labels:       <none>
Annotations:  <none>
Spec:
  PodSelector:     app=backend
  Allowing ingress traffic:
    To Port: 80/TCP
    From:
      PodSelector: app=frontend
  Not affecting egress traffic
  Policy Types: Ingress
```